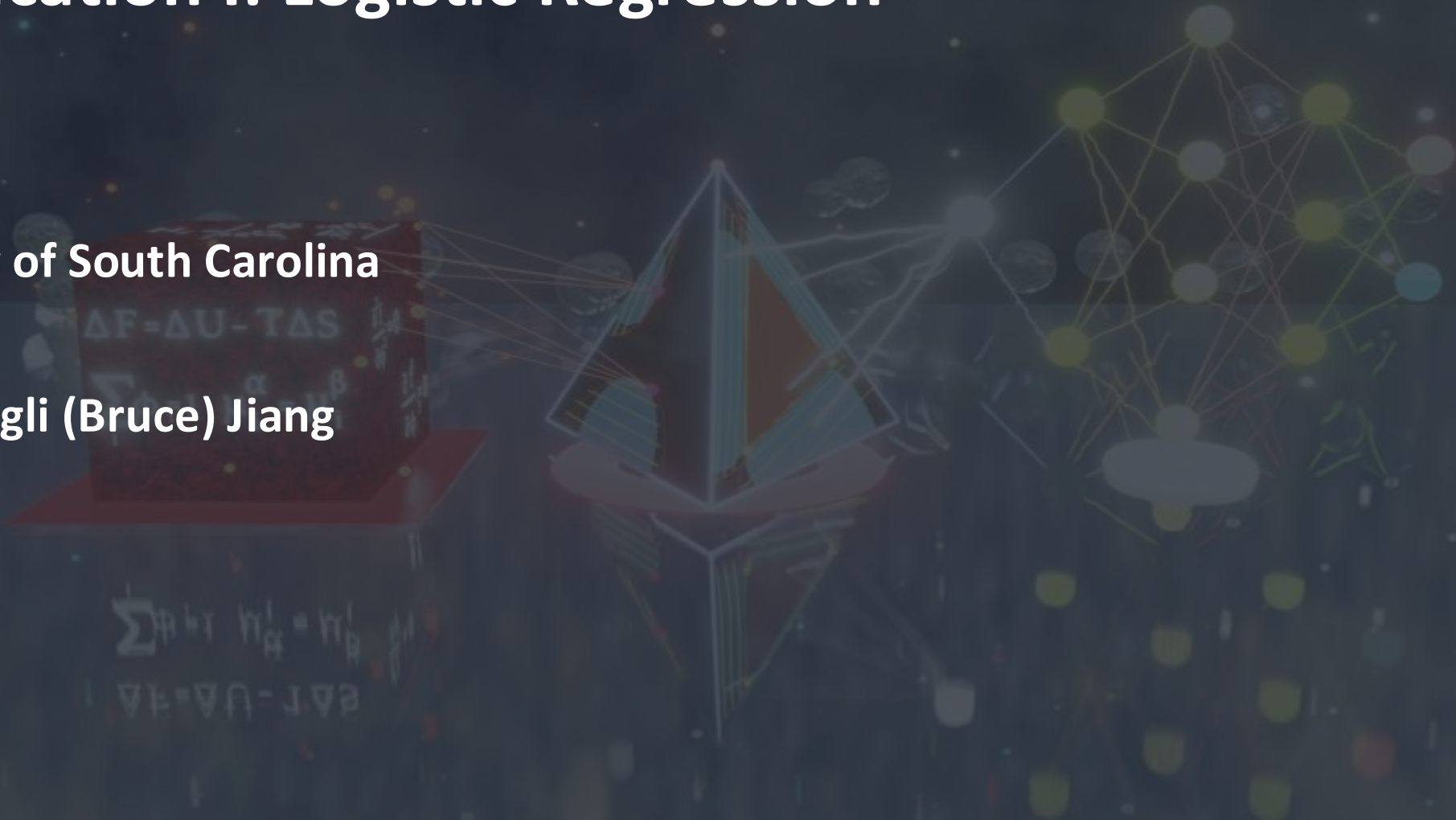


Classification I: Logistic Regression

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang



From Lecture 11

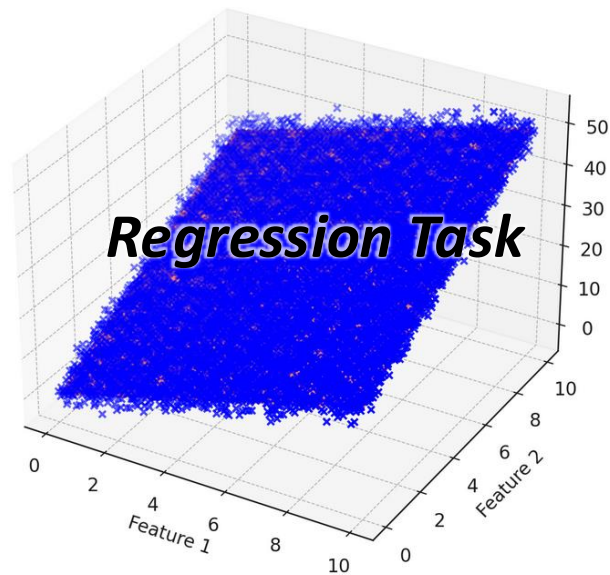
Categorical Features and Missing Values

- *One-hot encoding converts categories into binary features.*
- *Imputation handles missing feature values.*
- *Pipelines apply preprocessing without data leakage.*

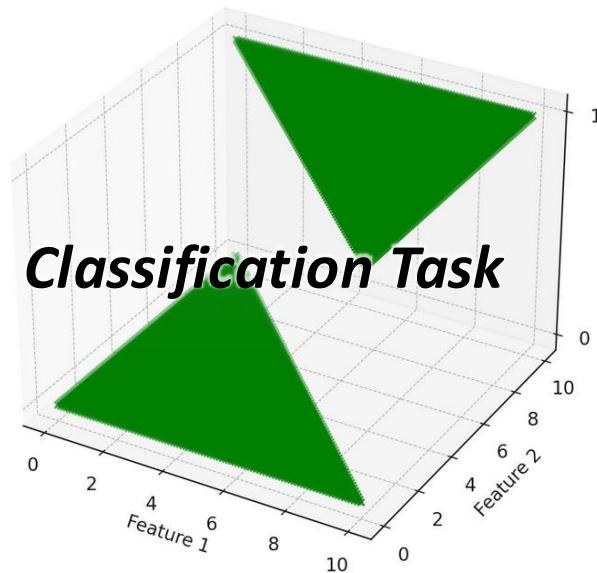
In this lecture, you will learn

- *Distinguish classification from regression.*
- *Explain logistic regression, probabilities, and log loss.*
- *Extend binary classification using one-vs-rest and one-vs-one.*

Classification vs. Regression

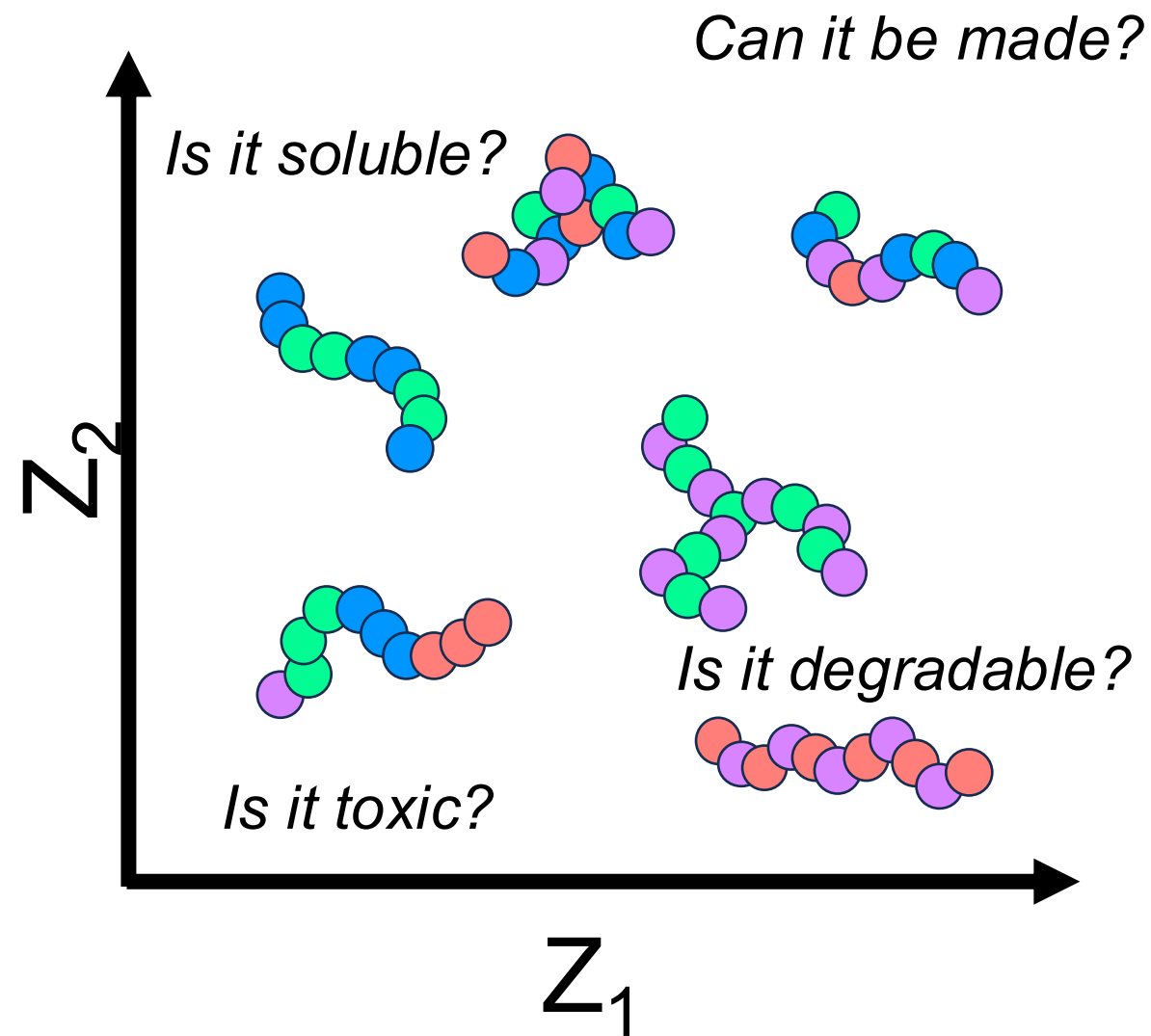
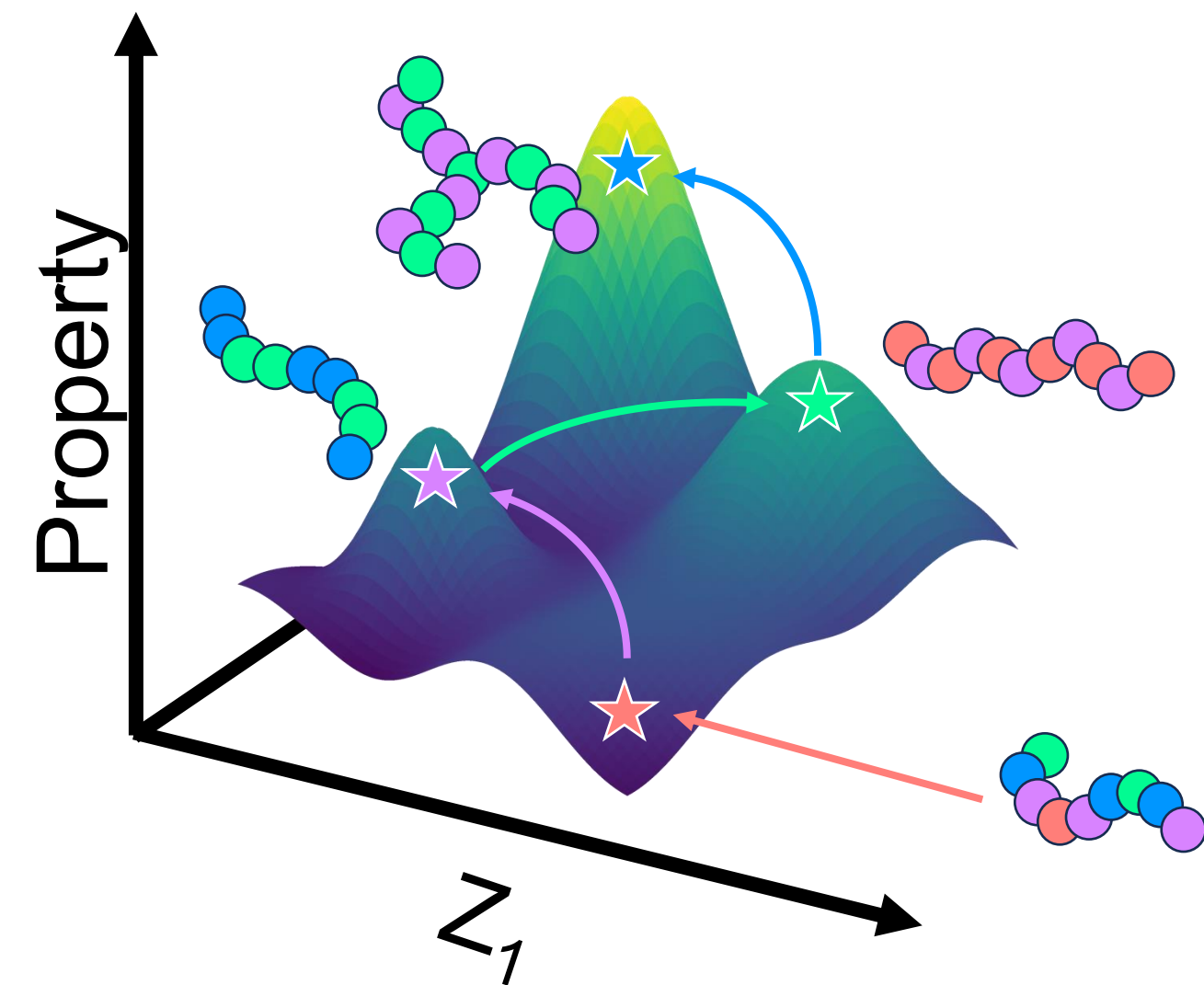


- **Output:** Predicts **continuous values**
- **Goal:** Estimate a numerical value based on input features
- **Examples:**
 - Predicting the **yield strength** of a new polymer based on its molecular structure.
 - Estimating the **reaction rate** of a chemical process at different temperatures.
 - Predicting the **lifetime** or **degradation rate** of a battery under various operating conditions.

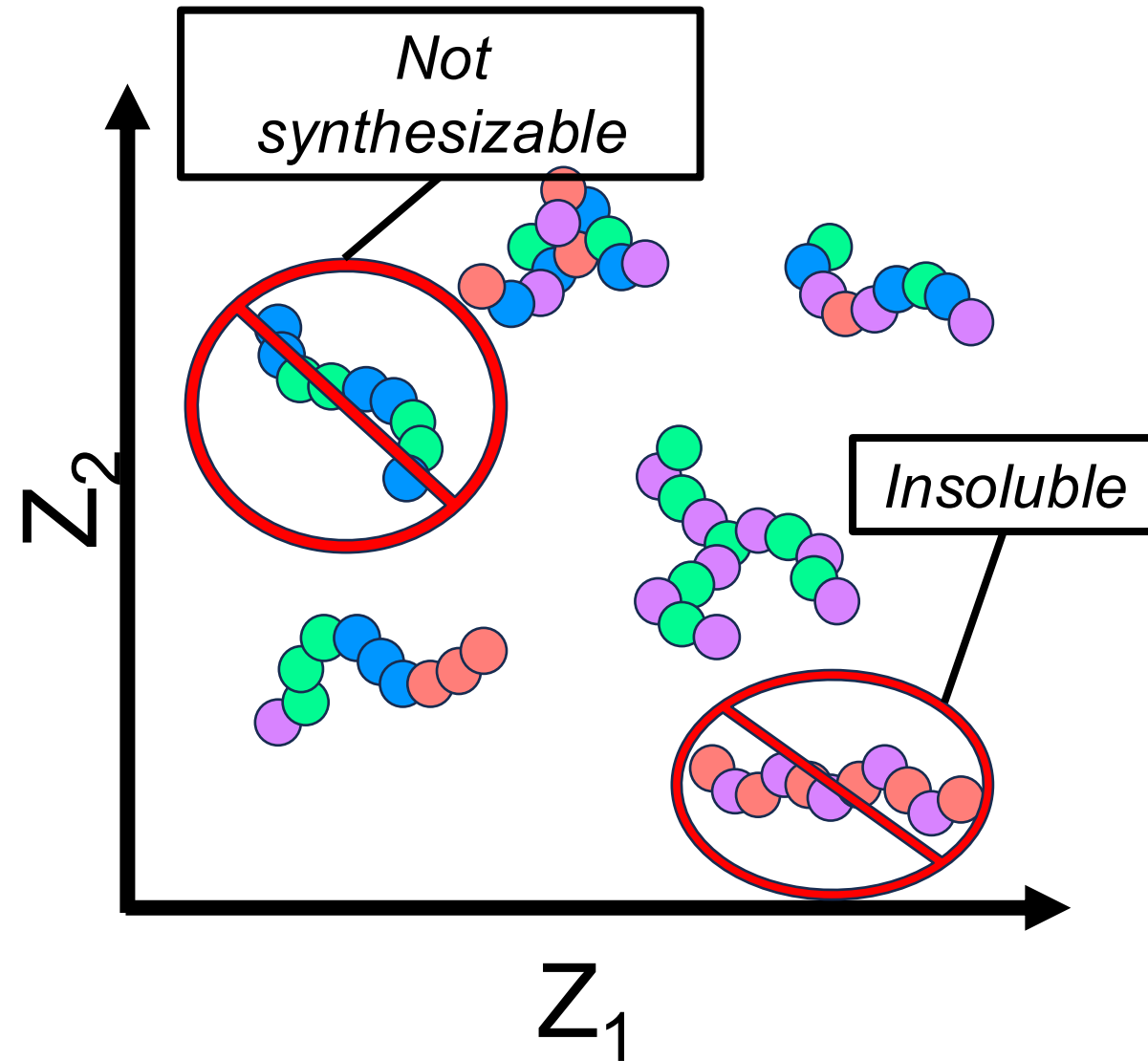
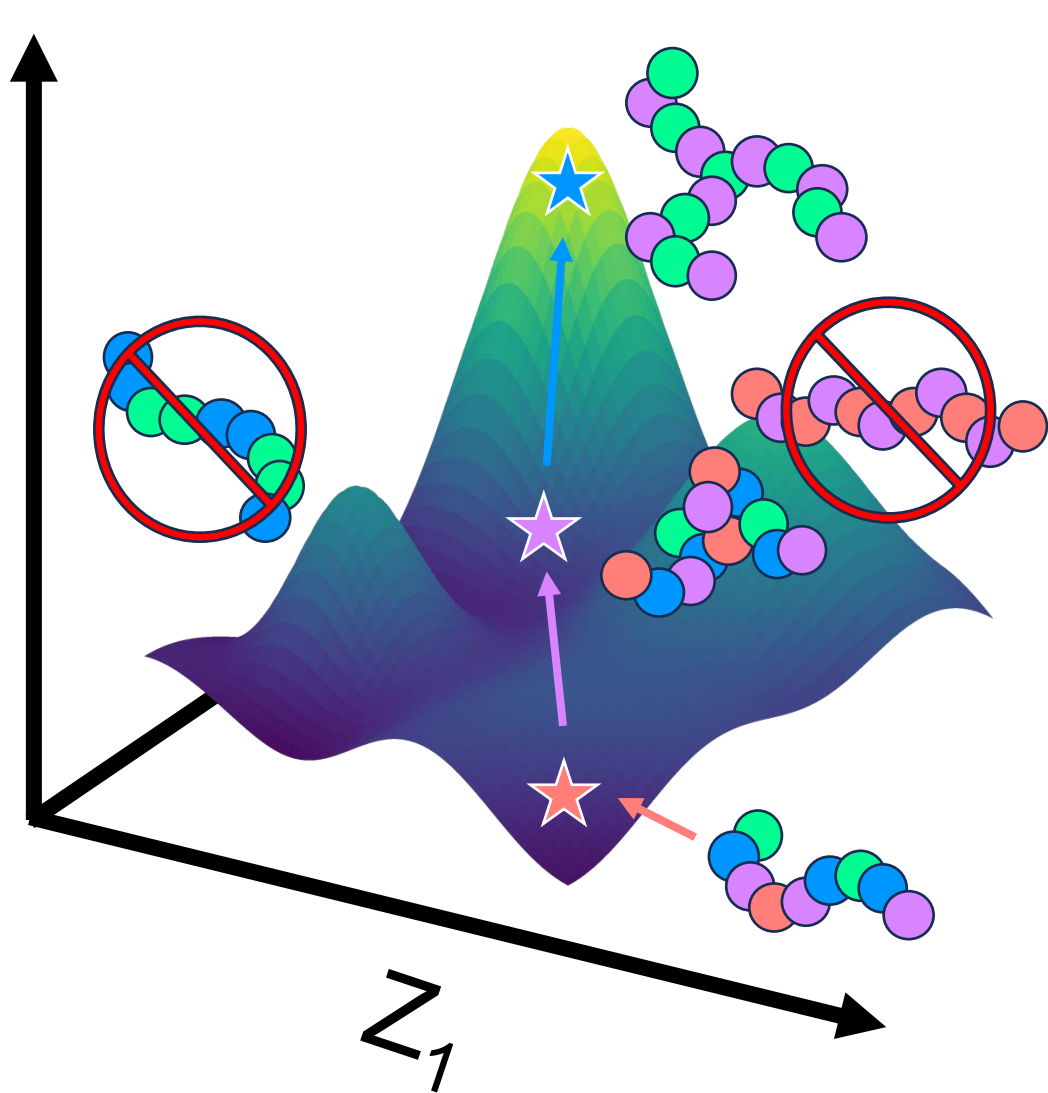


- **Output:** Predicts **discrete labels** or **categories**
- **Goal:** Assign input data to one of several predefined categories
- **Examples:**
 - Determining whether a chemical reaction will be **exothermic** or **endothermic** given certain reactants and conditions.
 - Classifying polymers as **soluble** or **insoluble** in a particular solvent.
 - Predicting whether a material will be **brittle** or **ductile** based on its microstructure.

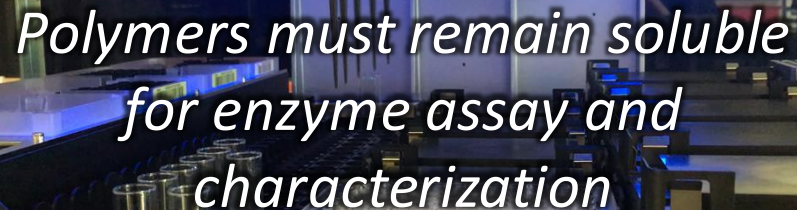
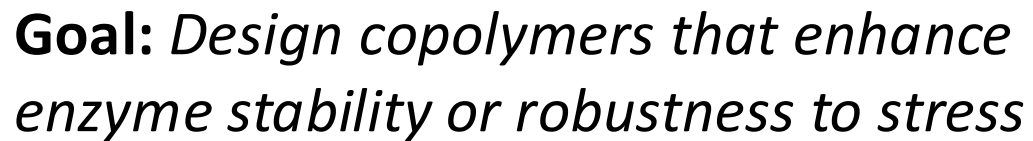
Classification in Chemical/Materials Optimization



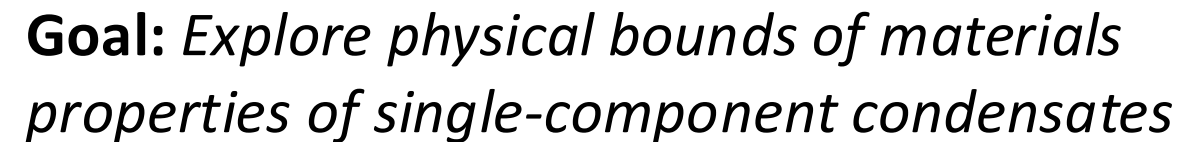
Classification in Chemical/Materials Optimization



composition design



sequence design



Which problem is classification?

- A. Predict methanol yield in grams.
- B. Predict whether a polymer is soluble.
- C. Predict catalyst lifetime in hours.

Logistic Regression: Pathway towards Classification

In *logistic regression*, we want to restrict our predictions to be on the interval $[0,1]$ to represent probabilities of a class.

A linear model first produces a score:

$$z = \mathbf{x}^T \boldsymbol{\theta}$$

- The score can range from $-\infty$ to $+\infty$.
- A probability must remain between 0 and 1.

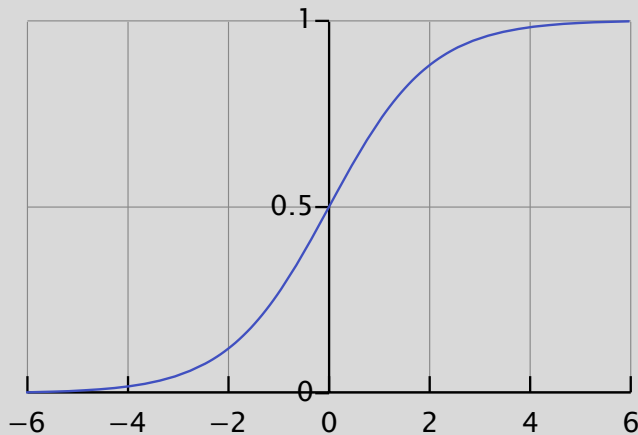
We need a function that converts an **unbounded linear score** into a **probability**.

Logistic Regression: Pathway towards Classification

In **logistic regression**, we want to restrict our predictions to be on the interval $[0,1]$ to represent probabilities of a class.

Logistic Function

$$f(x) = \frac{L}{1 + e^{-k(x-x_0)}}$$



$$L = 1, k = 1, x_0 = 0$$

other Sigmoid shapes can be used for analogous purpose

The essential premise of a **logistic model** is to represent the **log-odds** of a label as a linear combination of the features

$$\ell = \log_b \frac{p}{1-p} = \mathbf{x}^T \boldsymbol{\theta}$$

$$\Rightarrow p = \frac{1}{1 + b^{-\mathbf{x}^T \boldsymbol{\theta}}} \xrightarrow{b=e} \frac{1}{1 + e^{-\mathbf{x}^T \boldsymbol{\theta}}}$$

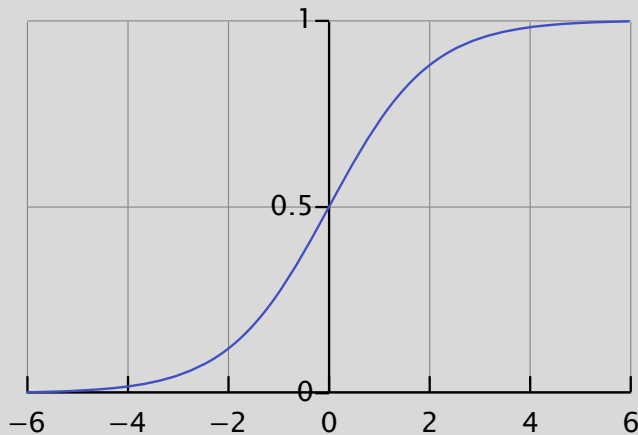
Model predictions: $\hat{y} \leftarrow f(x) = p(y = 1 | \mathbf{x}, \boldsymbol{\theta})$

Logistic Regression: Pathway towards Classification

In **logistic regression**, we want to restrict our predictions to be on the interval $[0,1]$ to represent probabilities of a class.

Logistic Function

$$f(x) = \frac{L}{1 + e^{-k(x-x_0)}}$$



$$L = 1, k = 1, x_0 = 0$$

other Sigmoid shapes can be used for analogous purpose

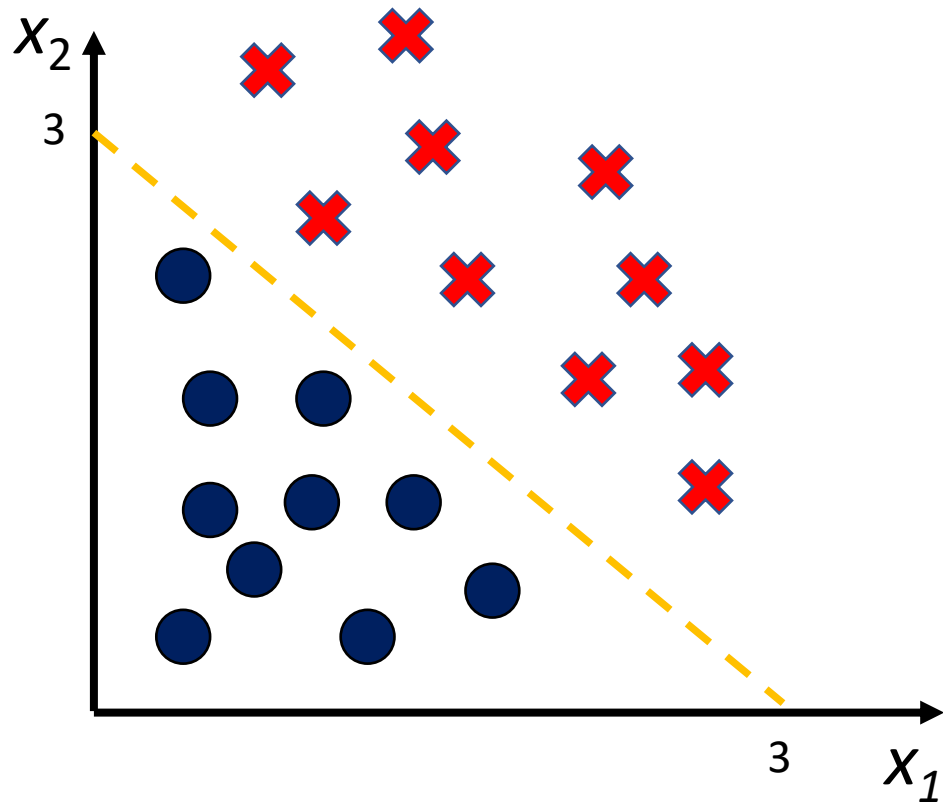
The essential premise of a **logistic model** is to represent the **log-odds** of a label as a linear combination of the features

$$\implies p = \frac{1}{1 + b^{-x^T \theta}} \xrightarrow{b=e} \frac{1}{1 + e^{-x^T \theta}}$$

- p is the probability of being 1.
- $z = x^T \theta = 0$ gives $p = 0.5$.
- Positive scores give $p > 0.5$.
- Negative scores give $p < 0.5$.

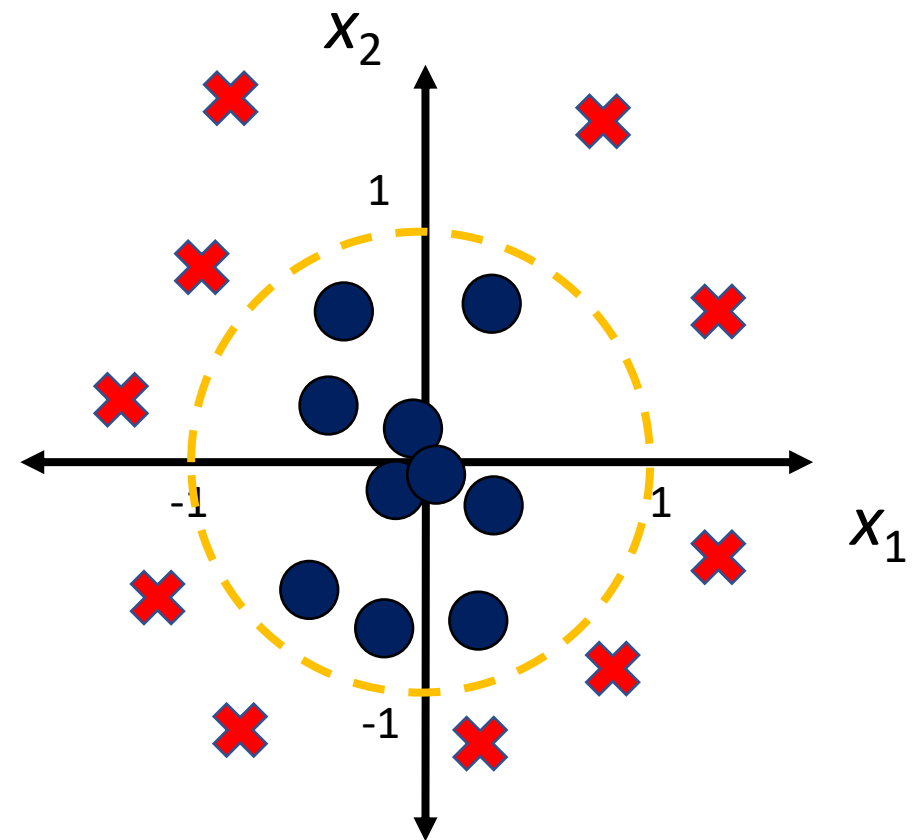
for $f(x) = 0.7$, we interpret that to mean a 70% chance that $y = 1$

Parameterizing Decision Boundaries



$$f(x_1, x_2) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2)}}$$

what would be a good set of thetas?



$$f(x_1, x_2) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)}}$$

what about in this case?

Approaching a Cost Function

As in linear regression, we will identify optimal parameters via minimization of an appropriate cost function; here, we have something to think about

Suppose model predictions are supplied via

$$\hat{y} \leftarrow f(x) = p(y = 1 | \mathbf{x}, \boldsymbol{\theta}) = \frac{1}{1 + e^{-\mathbf{x}^T \boldsymbol{\theta}}}$$

Can you anticipate any potential issues with our previous mean-squared error metric?

$$\mathcal{E}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

As an alternative, we might consider

$$\mathcal{E}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \varepsilon(\hat{y}_i, y_i)$$

$$\varepsilon(\hat{y}_i, y_i) = \begin{cases} -\log[f(\mathbf{x}_i)], & \text{if } y_i = 1 \\ -\log[1 - f(\mathbf{x}_i)], & \text{if } y_i = 0 \end{cases}$$

MSE vs. Binary Cross-Entropy

Can you anticipate any potential issues with our previous mean-squared error metric?

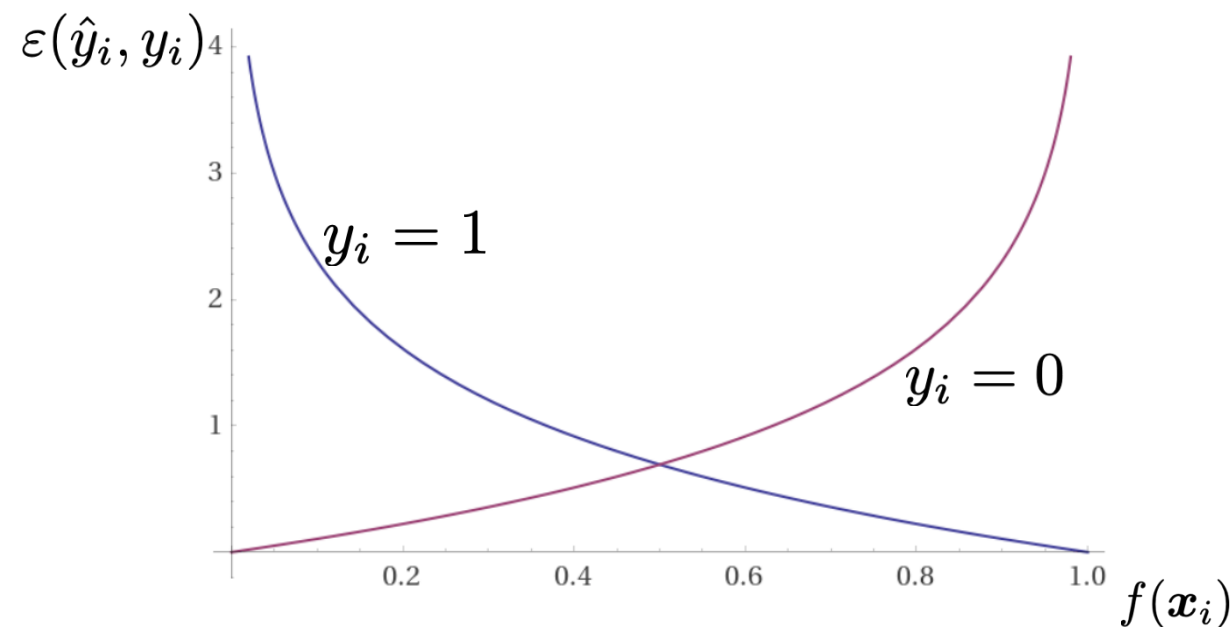
$$\mathcal{E}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

For $y = 1$:

Predicted $\hat{p}$	MSE	Log loss
0.90	0.01	0.105
0.10	0.81	2.303
0.01	0.98	4.605

$$\mathcal{E}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \varepsilon(\hat{y}_i, y_i)$$

$$\varepsilon(\hat{y}_i, y_i) = \begin{cases} -\log [f(\mathbf{x}_i)], & \text{if } y_i = 1 \\ -\log [1 - f(\mathbf{x}_i)], & \text{if } y_i = 0 \end{cases}$$



Cross-entropy gives a large penalty to confident mistakes and produces a well-behaved objective for logistic regression.

Minimizing the Cost Function

$$\varepsilon(\hat{y}_i, y_i) = -y_i \log f(\mathbf{x}_i) - (1 - y_i) \log [1 - f(\mathbf{x}_i)]$$

$$\mathcal{E}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \varepsilon(\hat{y}_i, y_i) \quad \hat{y} \leftarrow f(\mathbf{x}) = p(y = 1 | \mathbf{x}, \boldsymbol{\theta}) = \frac{1}{1 + e^{-\mathbf{x}^T \boldsymbol{\theta}}}$$

Suppose we were to use gradient descent

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \gamma_i [\nabla_{\boldsymbol{\theta}} \mathcal{E}(\boldsymbol{\theta})]$$

$\vdots$

$$\boldsymbol{\theta}_j \leftarrow \boldsymbol{\theta}_j - \gamma_i \sum_{i=1}^n [f(\mathbf{x}_i) - y_i] (\mathbf{x}_i)_j$$

*This is the same result
as for linear regression!*

For a sample with $y = 1$, the model predicts $p(y = 1) = 0.10$.

1. What class results from a threshold of 0.5?
2. Should the log loss be small or large?

Binary Classification using Logistic Regression

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
    log_loss
)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

wine = load_wine(as_frame=True)
X = wine.data[["alcohol", "color_intensity"]]
y = (wine.target == 0).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=26, stratify=y
)
```

Classification metrics

Wine classification dataset

Binary classification

```
model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression())
])

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Log loss:", log_loss(y_test, y_prob))

ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred,
    display_labels=["Other", "Cultivar 0"],
    cmap="Blues"
)

plt.show()
```

Classifier

Class

Probability

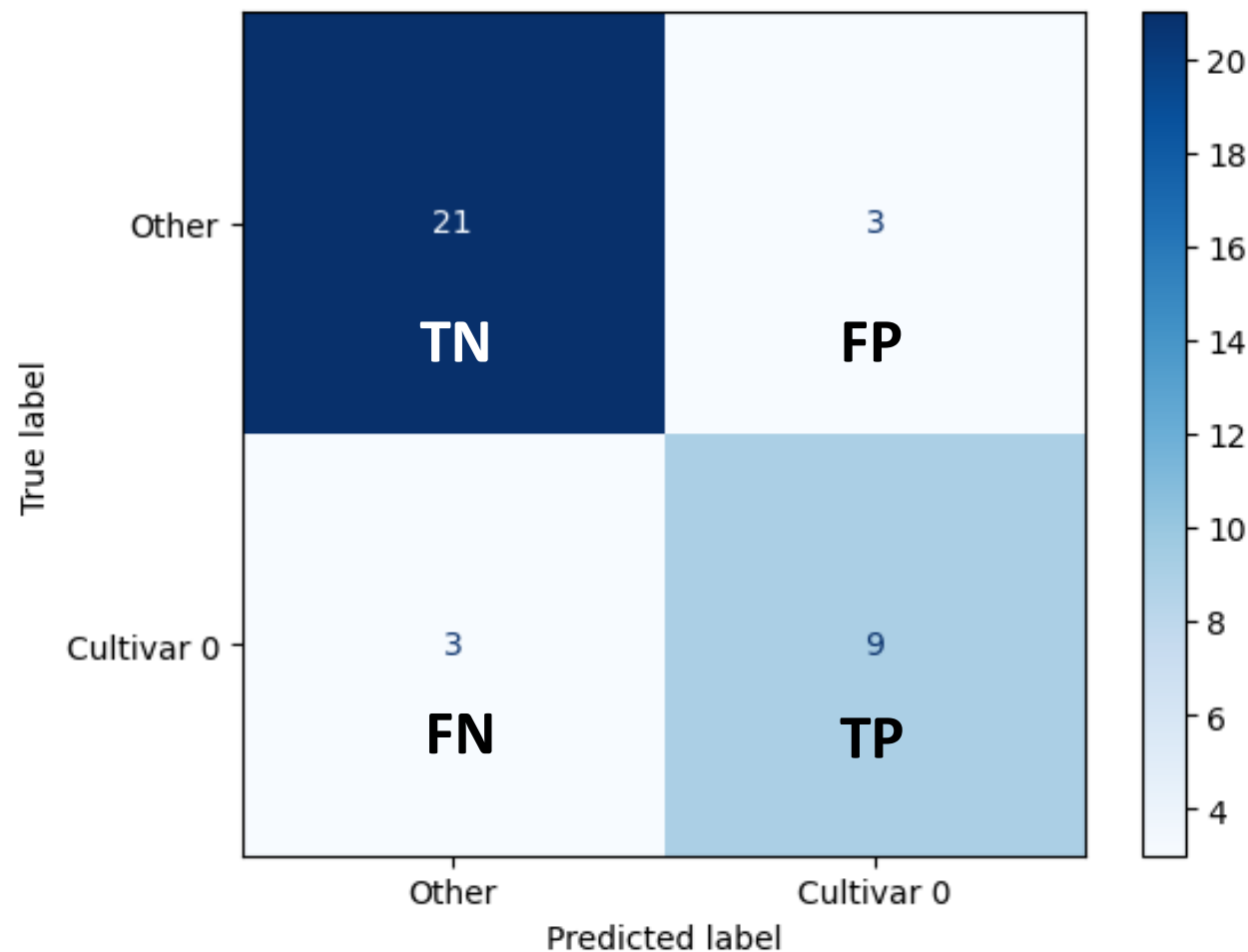
Binary Classification using Logistic Regression

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\hat{y}_i = y_i)$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

What is the accuracy score?

Assume **Cultivar 0** is the positive class.



Binary Classification using Logistic Regression

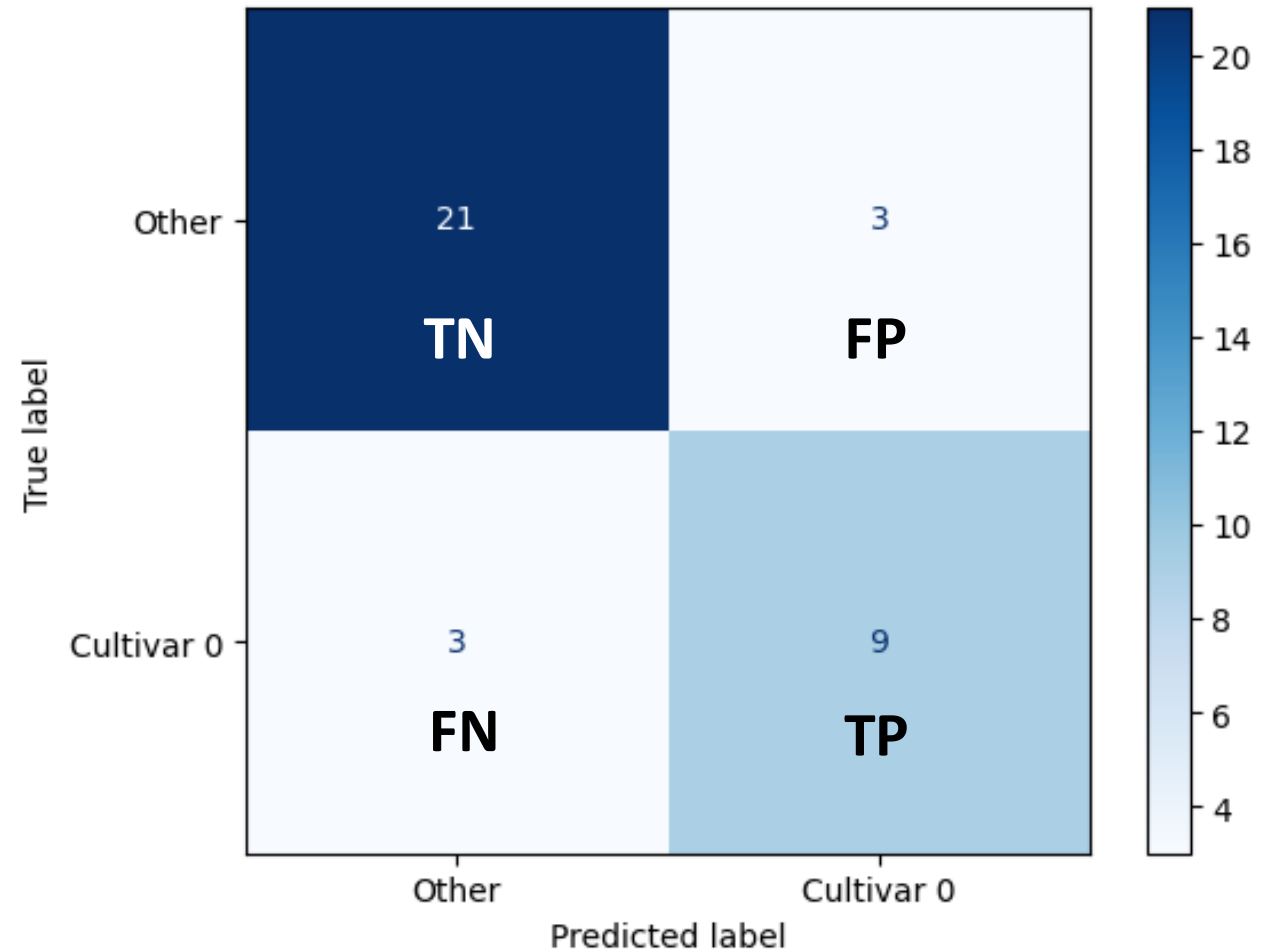
Precision measures how often a positive prediction is correct:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Use precision when **false positives are costly**. Here, it answers:

Of all samples predicted as Cultivar 0, what fraction actually belong to Cultivar 0?

Assume **Cultivar 0** is the positive class.



Binary Classification using Logistic Regression

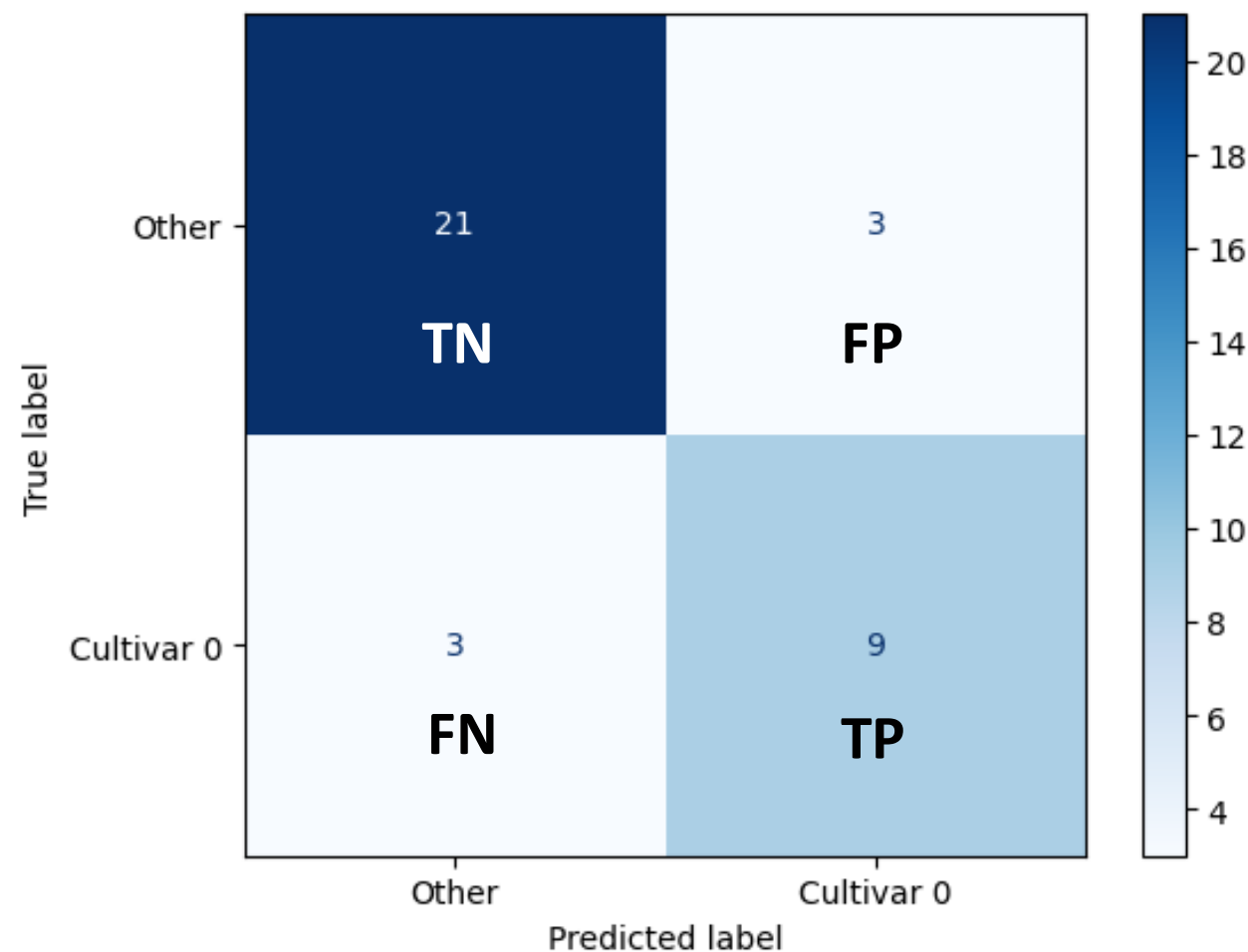
Recall measures how many actual positive samples are detected:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Use recall when **missing a positive sample is costly**. It answers:

Of all actual Cultivar 0 samples, what fraction did the model identify?

Assume **Cultivar 0** is the positive class.



Binary Classification using Logistic Regression

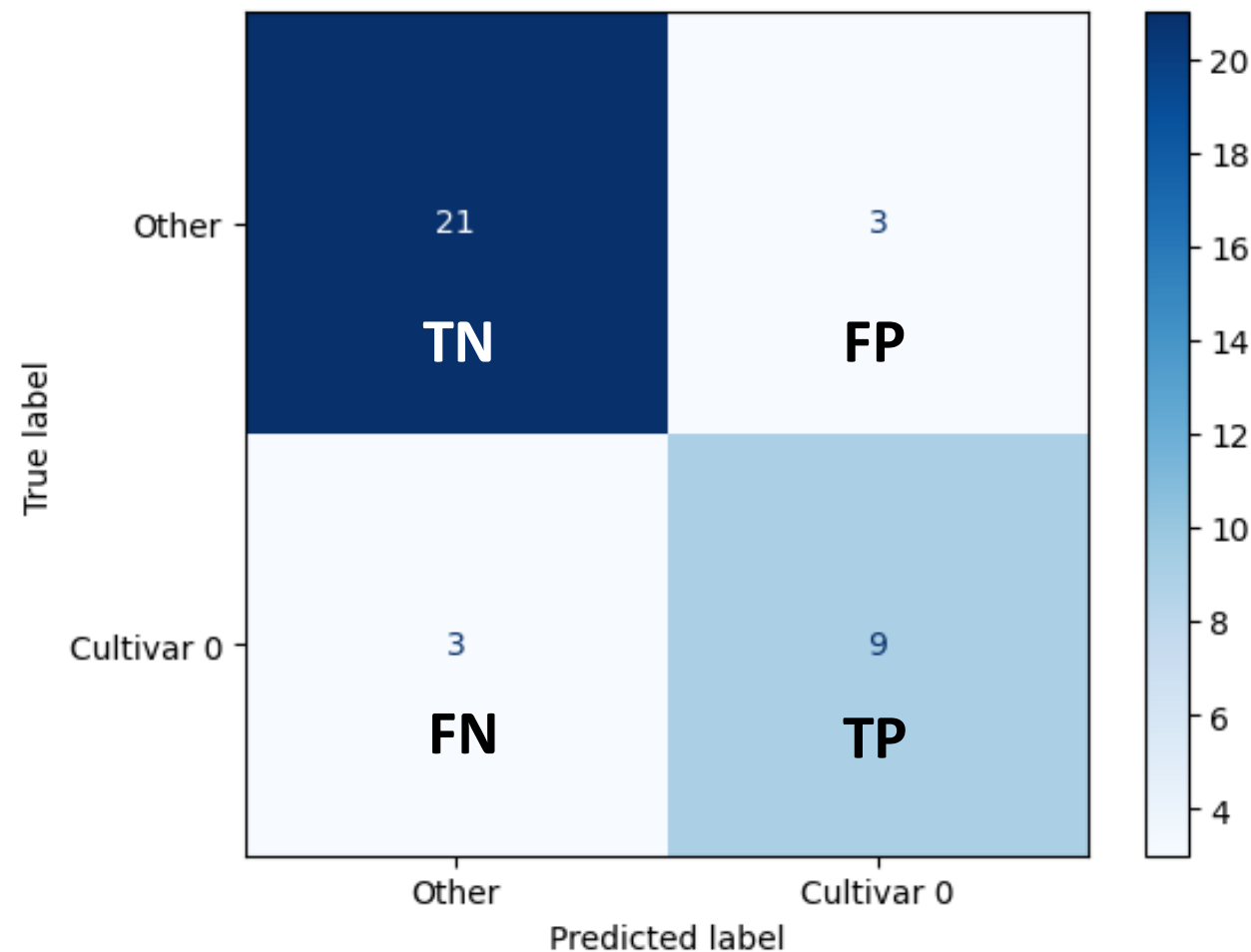
Specificity measures how well the model recognizes the negative class:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Use specificity when correctly rejecting negative samples matters. It answers:

Of all samples that are not Cultivar 0, what fraction did the model correctly reject?

Assume **Cultivar 0** is the positive class.



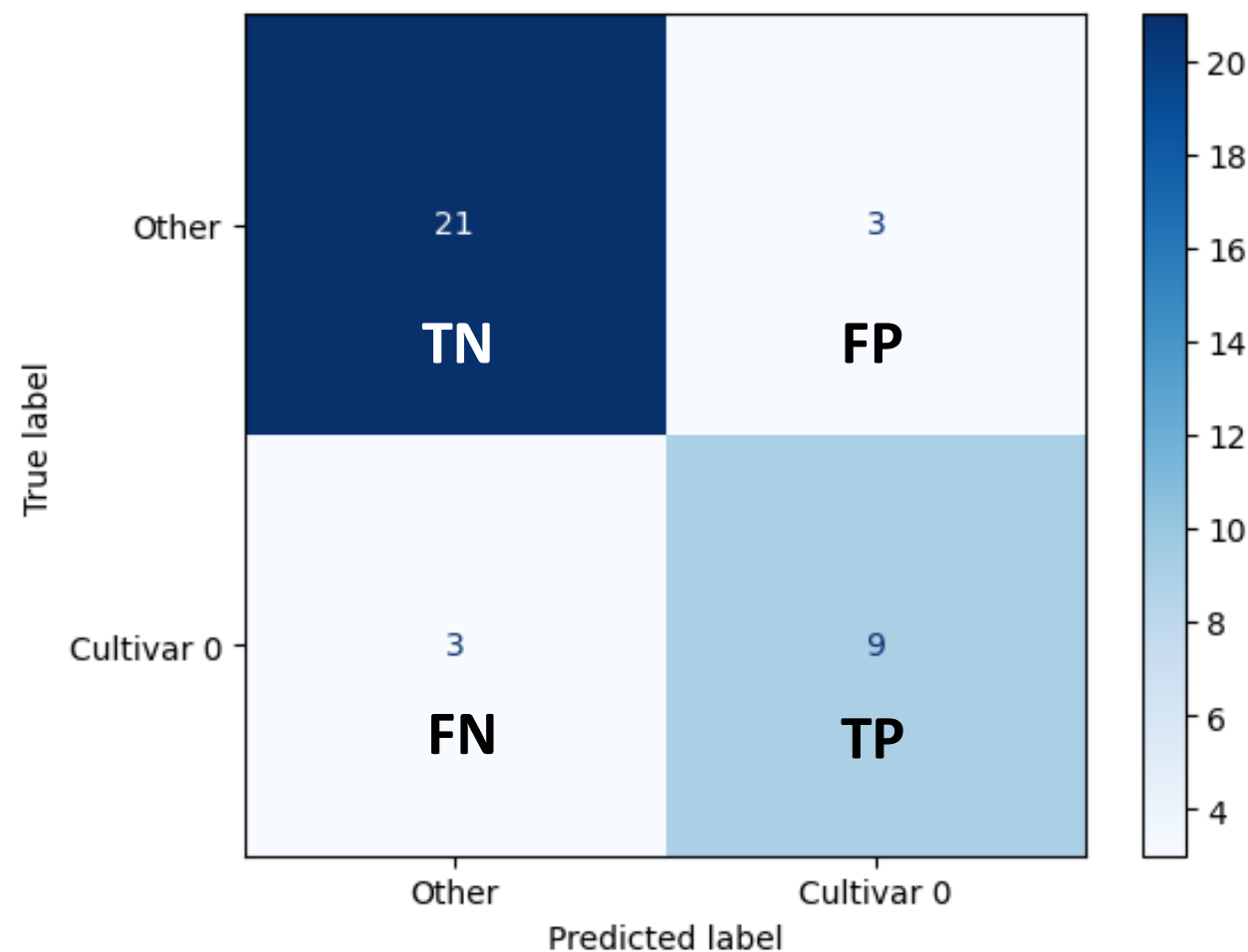
Binary Classification using Logistic Regression

The **F_1 score** is the harmonic mean of precision and recall:

$$F_1 = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$
$$= \frac{2TP}{2TP + FP + FN}$$

Use F_1 when you want one score that balances **false positives** and **false negatives**, particularly when the classes are imbalanced.

Assume **Cultivar 0** is the positive class.



Classification metrics

```
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score
)

y_pred = model.predict(X_test)

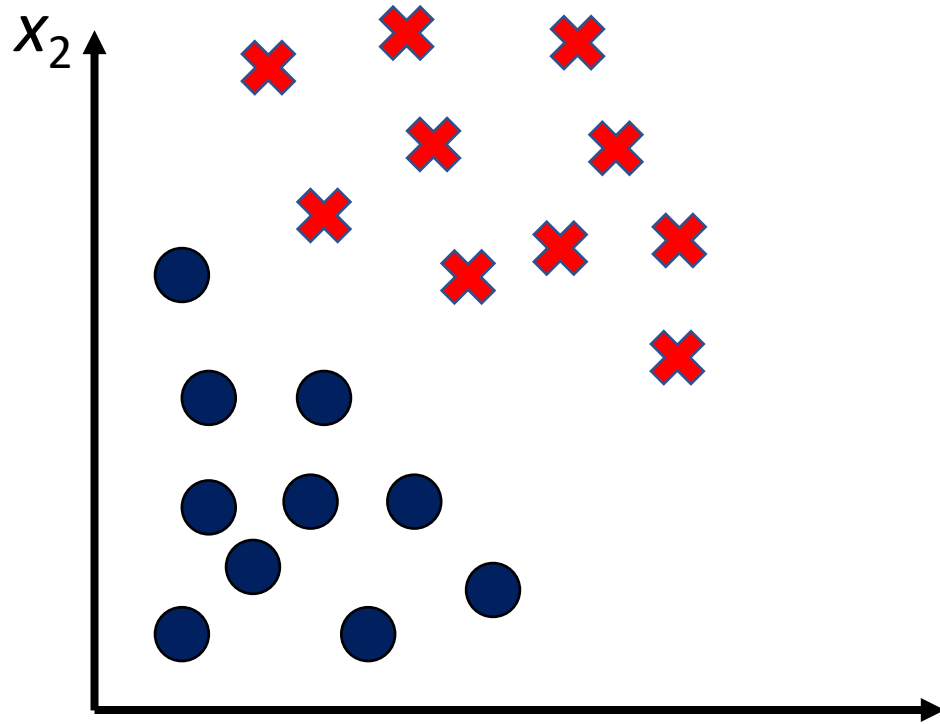
tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
specificity = tn / (tn + fp)
f1 = f1_score(y_test, y_pred)

print(f"Accuracy:      {accuracy:.3f}")
print(f"Precision:     {precision:.3f}")
print(f"Recall:         {recall:.3f}")
print(f"Specificity:    {specificity:.3f}")
print(f"F1 score:      {f1:.3f}")
```

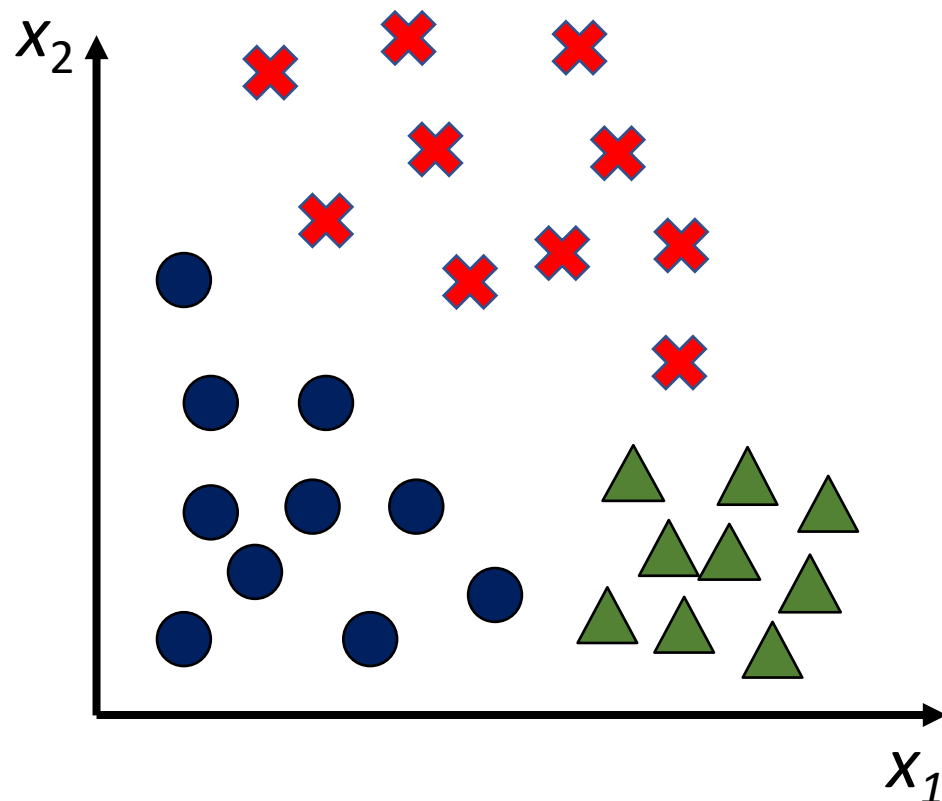
```
Accuracy:      0.833
Precision:     0.750
Recall:        0.750
Specificity:   0.875
F1 score:      0.750
```

Application now to classification: One vs. All



- **Binary case:** we just need to find the optimal decision boundary that partitions these classes

Application now to classification



- **Binary case:** we just need to find the optimal decision boundary that partitions these classes

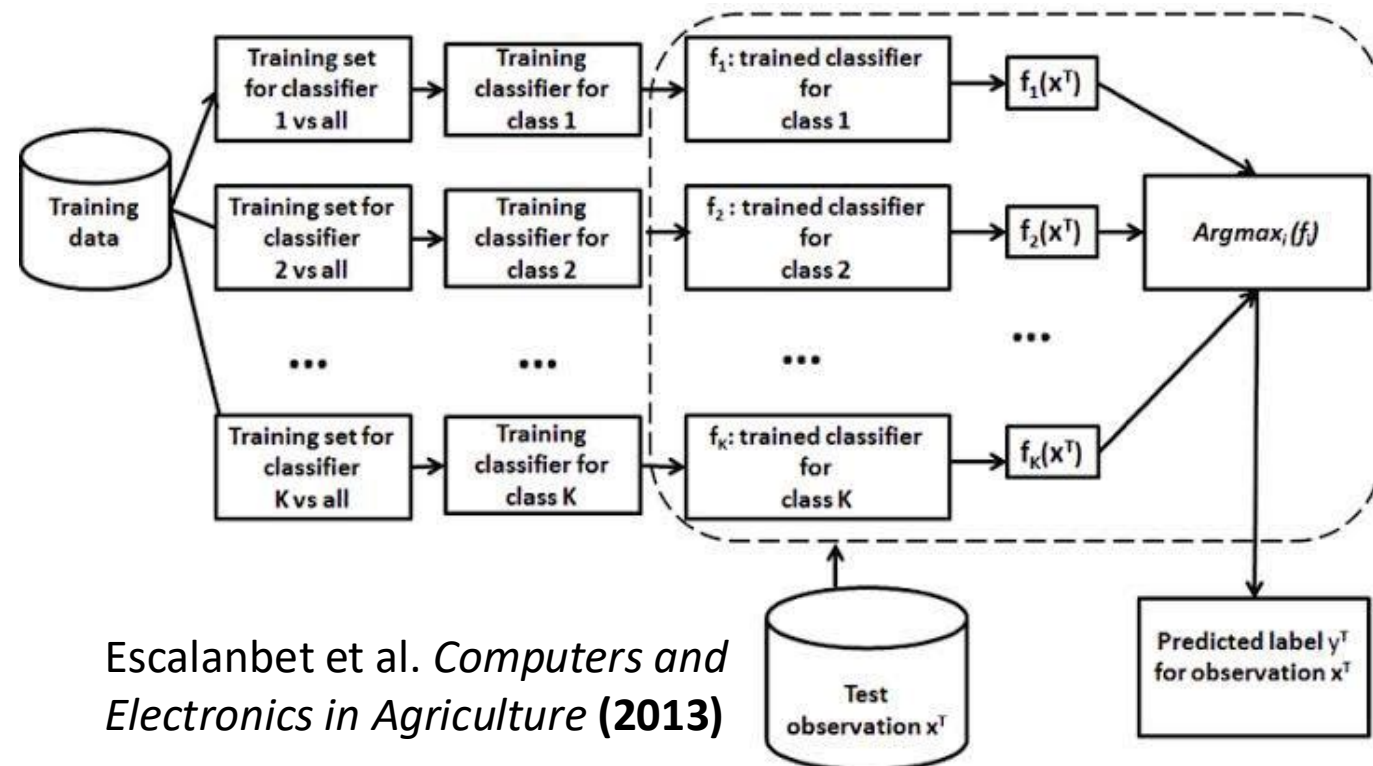
what do we do for multiple classes?

- **Multiclass case:** there are multiple strategies

Consider our data to have N classes...

One vs. All (Rest)

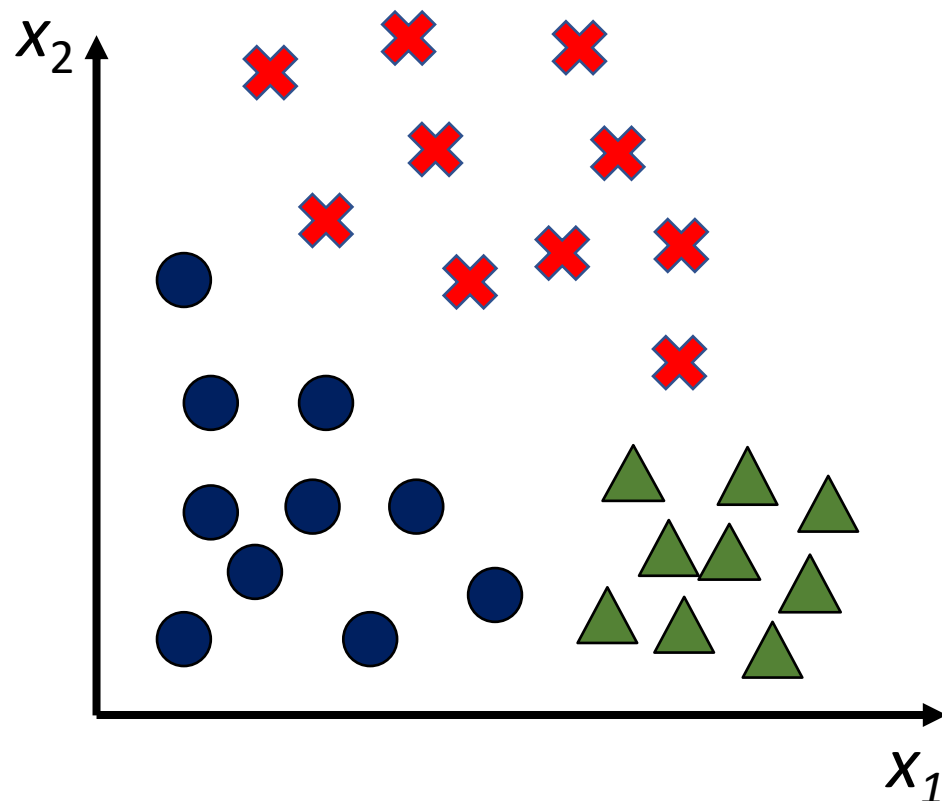
- formulate N binary classifier models



Escalanbet et al. *Computers and Electronics in Agriculture* (2013)

- pick the class that exhibits the highest score

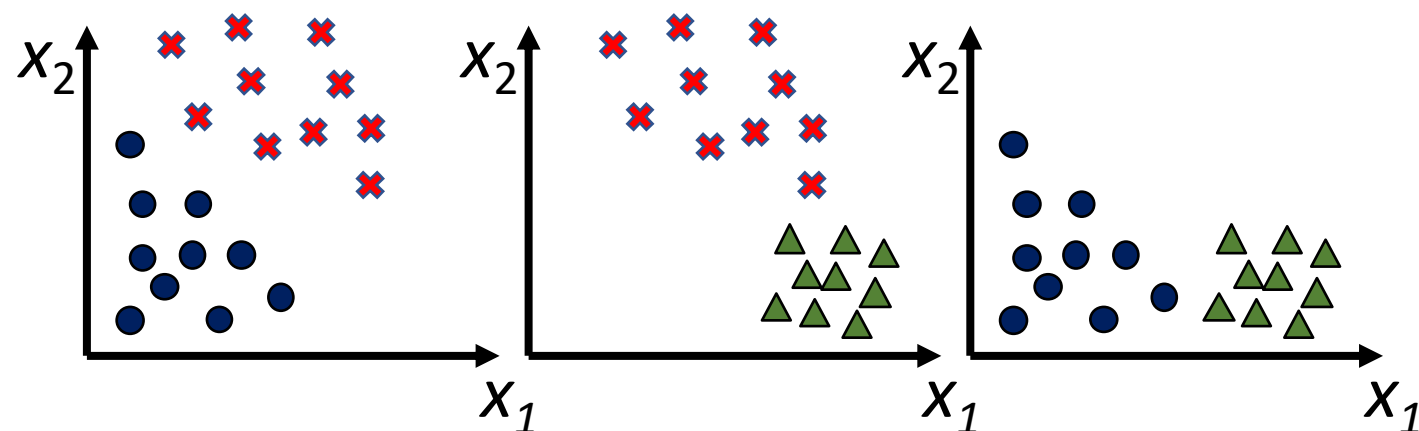
Application now to classification: One vs. One



Consider our data to have N classes...

One vs. One

- formulate $N(N-1)/2$ binary classifier models



- pick the class that receives the most positive identifications

- **Binary case:** we just need to find the optimal decision boundary that partitions these classes

what do we do for multiple classes?

- **Multiclass case:** there are multiple strategies

For four classes:

1. How many classifiers does one-vs-rest train?
2. How many does one-vs-one train?

Interactive Python Time

`12_classification_intro.ipynb`